

Voice AI Agent Patient Registration System

Take-Home Technical Assessment

Role:	Voice AI / Conversational AI Engineer
Time Limit:	3 Hours (Maximum)
Difficulty:	Intermediate — Advanced

CONFIDENTIAL — FOR CANDIDATE USE ONLY

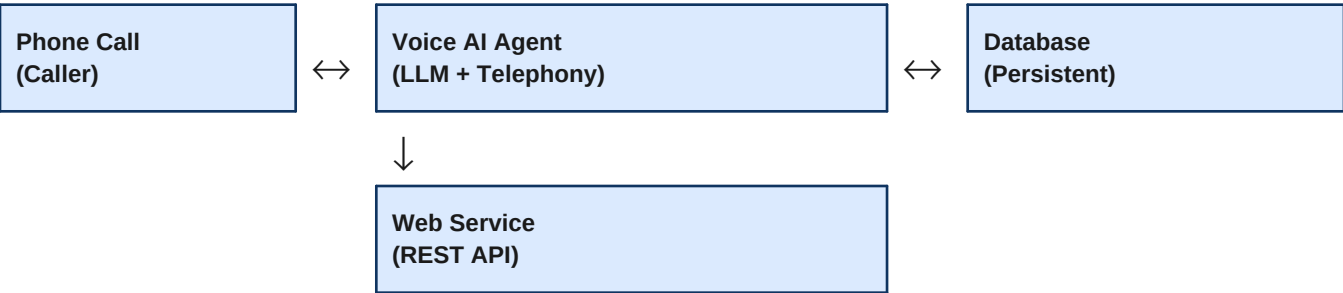
Overview

Build a **voice-based AI agent** accessible via a real phone number that collects standard U.S. patient demographic information through natural conversation, persists that data to a database, and exposes it through a lightweight web service. When we call your number, we should be able to speak naturally to register a new patient — and when we call back, the data from our previous call should still be there.

This challenge evaluates your ability to integrate multiple systems (telephony, LLM, database, API) into a cohesive, production-oriented solution under time pressure.

System Architecture

A caller dials a phone number. A voice AI agent answers, greets the caller, and conversationally collects all required patient demographics. Upon completion, the agent confirms the information back to the caller, saves the record to a persistent database, and ends the call. A companion REST API allows querying and viewing the stored patient records.



Functional Requirements

1. Telephony & Voice Agent

Requirement	Details
Phone Number	Provision a real, dialable U.S. phone number using a telephony provider (e.g., Twilio, Vonage, Vapi, Retell, Bland.ai).
Voice Interaction	The agent must conduct a natural, conversational flow — not a rigid IVR menu. It should feel like speaking with a human intake coordinator.
LLM-Powered	Use any LLM (OpenAI, Anthropic, Google, open-source). Must understand varied phrasing, ask clarifying questions, and handle corrections.
Confirmation	Before saving, the agent must read back all collected information and ask the caller to confirm or correct any field.
Error Handling	If the caller provides invalid data (e.g., a 3-digit phone number, a future date of birth), the agent must re-prompt specifically for that field.

Requirement	Details
Call Completion	After successful registration, provide a brief confirmation (e.g., "You're all set, [First Name].") and end gracefully.

2. Patient Demographic Data Model

Collect and store the following fields, representing the **standard minimum demographic dataset** required by U.S. healthcare providers for patient registration:

Field	Type	Validation Rules	Req
first_name	String	1–50 chars, alphabetic + hyphens/apostrophes	Yes
last_name	String	1–50 chars, alphabetic + hyphens/apostrophes	Yes
date_of_birth	Date	Valid date, not in future, MM/DD/YYYY	Yes
sex	Enum	Male, Female, Other, Decline to Answer	Yes
phone_number	String	Valid U.S. 10-digit phone number	Yes
email	String	Valid email format	No
address_line_1	String	Street address	Yes
address_line_2	String	Apt/Suite/Unit if applicable	No
city	String	1–100 characters	Yes
state	String	Valid 2-letter U.S. state abbreviation	Yes
zip_code	String	5-digit or ZIP+4 U.S. format	Yes
insurance_provider	String	Name of insurance company	No
insurance_member_id	String	Alphanumeric member/subscriber ID	No
preferred_language	String	Default: English	No
emergency_contact_name	String	Full name	No
emergency_contact_phone	String	Valid U.S. 10-digit phone number	No
created_at	Timestamp	Auto-generated at creation (UTC)	Auto
updated_at	Timestamp	Auto-generated on modification (UTC)	Auto
patient_id	UUID	Auto-generated unique identifier	Auto

Conversational Note: The agent does NOT need to ask for every optional field on every call. It should collect required fields, then offer: "I can also collect your insurance information, emergency contact, and preferred language. Would you like to provide any of those?" — letting the caller opt in.

3. Persistent Database

Requirement	Details
Engine	Any relational or document database (PostgreSQL, SQLite, MySQL, MongoDB, etc.)
Persistence	Data must survive server restarts. If we register "Jane Doe" on Call 1, she must exist when we query on Call 2.
Schema	Must enforce the data model above with proper column types and constraints.
Seed Data	Optionally include 1–2 seed patient records for demonstration.

4. Web Service (REST API)

Expose the following endpoints:

Method	Endpoint	Description
GET	/patients	List all patients. Support optional query params: ?last_name=, ?date_of_birth=, ?phone_number=
GET	/patients/:id	Retrieve a single patient by patient_id (UUID).
POST	/patients	Create a new patient. Returns the created record with patient_id.
PUT	/patients/:id	Update an existing patient record. Partial updates allowed.
DELETE	/patients/:id	Soft-delete a patient record (set deleted_at timestamp; do not hard-delete).

API Standards:

- Return proper HTTP status codes (200, 201, 400, 404, 422, 500).
- Validate all inputs server-side (do not rely solely on the voice agent for validation).
- Return JSON responses with consistent envelope: { "data": {...}, "error": null }

5. Voice Agent ↔ Database Integration

The voice agent **must use the REST API** (or directly invoke the same service layer) to persist patient records:

- When the caller confirms their information, the agent triggers a POST /patients (or equivalent DB write).
- The agent should relay the outcome back to the caller: success confirmation or a graceful error message if the write fails.
- **Bonus:** If the caller provides a phone number that matches an existing patient, the agent should recognize this and ask: "It looks like we already have a record for [First Name] [Last Name]. Would you like to update your information instead?"

Non-Functional Requirements

Area	Expectation
Deployment	The system must be running and callable at the time of review. We will call the number. Use any hosting you prefer (Railway, Render, Fly.io, Replit, ngrok, AWS, GCP, etc.).
Code Quality	Clean, readable, well-organized code. We don't expect perfection in 3 hours, but we do expect intentional structure.
README	A clear README.md with: setup instructions, architecture description, tech stack justification, env variables required, known limitations or trade-offs.
Security	Do not hardcode API keys in source code. Use environment variables. Basic input sanitization on the API.
Observability	Log agent conversations (at minimum, the final collected data payload) to stdout or a log file.

Evaluation Criteria

Your submission will be scored across five dimensions, **weighted equally at 20% each**.

1. Working System (20%)

Does it work end-to-end?

- Can we call the number and complete a patient registration?
- Is the data actually persisted and retrievable via the API?
- Does the system handle a second call without data loss?

2. Conversational Quality (20%)

Is the voice agent actually good?

- Does it sound natural, not robotic or scripted?
- Does it handle corrections gracefully? ("Actually, my last name is spelled D-A-V-I-S, not D-A-V-I-E-S")
- Does it confirm information before saving?
- Does it handle interruptions or out-of-order responses?

3. Technical Architecture (20%)

Are the right choices being made?

- Is the separation of concerns clear (telephony vs. LLM logic vs. data layer vs. API)?
- Is the database schema well-designed with appropriate types and constraints?
- Are API endpoints RESTful and properly validated?
- Is the prompt engineering for the voice agent thoughtful and documented?

4. Code Quality & Documentation (20%)

Could another engineer pick this up?

- Is the code organized, readable, and consistent?
- Is the README complete and accurate?
- Are trade-offs and known limitations documented?
- Is the prompt/system message for the LLM included and commented?

5. Edge Cases & Resilience (20%)

What happens when things go wrong?

- What if the caller says an invalid date of birth?
- What if the telephony connection drops mid-call?
- What if the database write fails — does the caller get an error or silence?
- What if the caller wants to start over mid-conversation?

Bonus Challenges (Not Required)

These do not affect your core score but demonstrate depth and will be noted positively:

- **Duplicate Detection:** Agent recognizes returning callers by phone number and offers to update instead of create.
- **Appointment Scheduling:** After registration, offer to schedule a first appointment (mock data is fine).
- **Multi-language Support:** If the caller says "Hablo español," the agent switches to Spanish.
- **Call Recording/Transcript:** Store a transcript or summary of each call linked to the patient record.
- **Dashboard:** A simple web UI that displays registered patients from the database.
- **Automated Tests:** Unit or integration tests for the API layer.

Recommended Tech Stack (Suggestions Only)

You may use any technologies you are comfortable with. Here are common combinations that work well within the time constraint:

Layer	Options
Telephony + Voice AI	Vapi, Retell AI, Bland.ai, Twilio + Deepgram/ElevenLabs, Vonage
LLM	OpenAI GPT-4o / 4o-mini, Anthropic Claude, Google Gemini, Groq + Llama
Backend	Node.js (Express/Fastify), Python (FastAPI/Flask), Go, Ruby on Rails

Layer	Options
Database	PostgreSQL, SQLite (for simplicity), MongoDB, Supabase
Hosting	Railway, Render, Fly.io, Replit, Vercel + serverless DB, ngrok (for local dev)

Tip: Platforms like Vapi and Retell abstract much of the telephony/STT/TTS complexity and let you focus on the LLM prompt, tool definitions, and backend. This is the fastest path to a working system within 3 hours.

Submission Instructions

- Repository:** Push your code to a public or private GitHub/GitLab repository. If private, grant access to the reviewer emails provided separately.
- Live Demo:** The phone number and API endpoint must be live and accessible at the time of review. Include both in your README.
- Deadline:** Submit within **3 hours** of receiving this challenge. Partial submissions are accepted — a working but incomplete system scores higher than a non-functional but ambitious one.
- Send us:**
 - Repository URL
 - Phone number to call
 - API base URL (e.g., <https://your-app.railway.app>)
 - Any credentials or notes needed for testing

What We Are Really Looking For

This is not a trick question. We are not looking for a production healthcare system with HIPAA compliance and 99.99% uptime.

We are looking for engineers who can:

- Integrate multiple systems** under time pressure without getting stuck.
- Make smart trade-offs** — knowing when to use a shortcut (SQLite over Postgres, ngrok over cloud deploy) and when to invest (clean prompt engineering, proper error handling).
- Build things that work** — end to end, not just in isolation.
- Think about the user experience** — in this case, the person on the phone. A technically perfect system with a terrible voice experience is a failure.
- Communicate clearly** — through code, documentation, and architectural decisions.

We would rather see a simple system that works flawlessly than an over-engineered system that crashes on the first call.

FAQs

Q: Can I use a voice AI platform (Vapi, Retell, etc.) instead of building STT/TTS from scratch?

A: Absolutely. In fact, we encourage it. The goal is to assess your integration and system design skills, not your ability to implement a speech-to-text engine.

Q: Do I need to handle HIPAA compliance?

A: No. This is a technical assessment, not a production healthcare system. Do not store real patient data.

Q: What if I can't get a phone number provisioned in time?

A: Document what you tried and why it failed. Provide a working local setup with clear instructions for us to test. You will not be penalized for vendor issues, but you will be evaluated on how you handled the blocker.

Q: Can I use AI coding assistants (Copilot, Cursor, Claude, etc.)?

A: Yes. We care about the output, your understanding of it, and your ability to explain your decisions — not whether you typed every character.

Q: What if I run out of time?

A: Submit what you have. Document what you would have done next in your README under a "Next Steps" section. Partial, working submissions are valued.

Good luck. We look forward to calling your agent.